CivicConnect staff view

What was built, which documents it follows, and how the code is put together.

Project: SEN381 Group E, CivicConnect. Date: 13 October 2026.

1. What this note is for

This note explains the staff side of CivicConnect. It says what the requirements ask staff to be able to do, what was added to the program, and how that code follows the same structure and the same visual design as the rest of the application.

The staff side is two screens inside the existing signed-in shell: a request list, and the detail of one request. Staff reach the detail by choosing a request on the list. That is one step from the list, which satisfies the rule that full details must be reachable within two navigation steps of the list (FR007).

2. What the documentation requires

The content of the staff view comes from the functional requirements FR005 to FR011 in docs/requirements/functional-requirements.md, and from the status rules D2 to D6 in docs/desicions/D2-D6-status-transition-authority.md. The role filter follows ADR-DESIGN-002 (one filter strategy per role). The allowed status steps follow ADR-DESIGN-001 and the StatusTransitionGuard class that was already in the backend.

Each requirement, and where it shows up on screen:

FR005

Staff see only requests in their authorised scope.

The list is loaded from the server already filtered. It is not the full database.

FR006

Staff can search, filter or sort by status, category or date.

The list has a search box, a status filter, a category filter, and a newest/oldest date sort.

FR007

Staff can open full details, including status and action history.

Choosing a row opens the detail screen: title, description, category, dates, submitter, assignee, status history and comments.

FR008

Staff can accept a request. Exactly one person is responsible.

On an Open request the button says Accept request. That assigns it to the signed-in staff member and shows the name.

FR009

Status may only move Open, Assigned, In Progress, Resolved, Closed.

The screen offers only the next legal step. Skipping a step is rejected by the server and shown as an error.

FR010

Staff can record a comment or resolution note.

The detail screen has a comment box. Saved comments stay in the list on that request.

FR011

Resolve and Close are separate steps, and only authorised staff may do them.

Mark resolved appears only at In Progress. Close request appears only at Resolved. Only the assigned staff member sees those buttons.

3. The status rules the buttons follow

D2 says who may move a request. Any staff member may take an Open request to Assigned. After that, only the assigned staff member (or Management) may move it to In Progress, then Resolved, then Closed. Any other jump is illegal.

D3 says an illegal jump returns HTTP 400, writes no history row, and does not change the status. D4 says assignment happens only while the request is Open, and it uses the same transition check. The clarification recorded with D2 to D6 says a staff member may assign a request only to themselves. The Accept button does that: it never asks for another person's name.

The screen therefore never shows a status dropdown. A dropdown would invite someone to pick Closed while the request is still Open. Instead there is one button, and its label changes with the current status:

- Open: Accept request. This assigns the request to you and sets the status to Assigned.
- Assigned, and you are the assignee: Start work. Status becomes In Progress.
- In Progress, and you are the assignee: Mark resolved.
- Resolved, and you are the assignee: Close request.
- Closed: no status button. The history and comments remain visible.

If the server still receives an illegal status, it answers with a sentence such as "Cannot change status from Open to Closed. The next status is Assigned." The detail screen shows that sentence in the same red alert used by the submit form.

4. What "authorised scope" means for staff

FR005 says the list is filtered to the staff member's authorised scope. ADR-DESIGN-002 says that rule lives in StaffFilterStrategy, not in an if-statement inside the controller.

The scope used here is: every Open request, plus every request already assigned to the signed-in staff member. Open requests have to be visible, otherwise nobody could accept them (FR008 and D2). Another staff member's Assigned, In Progress, Resolved or Closed work is left out, because D4 says only the assignee moves that request forward.

A request that is outside that scope is treated as not found if someone requests it by number. The list and the detail screen use the same check, so the detail page cannot be used to read another staff member's assigned work.

Search, status, category and date sorting happen in the browser, on that already-filtered list. The server's job for FR005 is to return the authorised set. The screen's job for FR006 is to let the staff member narrow and order that set. The category list is the same controlled list the submit form uses.

5. How the code follows the rest of the application

The application is already split the same way on both sides. The staff work uses those folders. It does not add a new framework, a router, or a second stylesheet.

Frontend, which matches src/README.md and the requester submit screen:

- src/frontend/views/StaffRequestList.jsx is the list screen. Screens live in views/.
- src/frontend/views/StaffRequestDetail.jsx is the detail screen.
- src/frontend/App.jsx still owns navigation. There is no React Router. A staff member gets a Requests item in the same nav bar. Choosing a row sets the screen to the detail, the same way Submit a request switches the requester screen.
- src/frontend/services/client.js holds the HTTP calls, using the same apiRequest helper, session cookie and /api/v1 base path as login and submit.
- src/frontend/services/validation.js checks the comment the same way it checks the submit form: trim, required, maximum length, field error.
- src/frontend/services/format.js turns the stored status InProgress into the label In Progress, and formats dates. The API still sends the enum names the database already uses.

Backend, which matches the package layout in src/README.md:

- ServiceRequestController in api/ exposes the HTTP paths. It does not decide who may see a request.
- ServiceRequestService in services/ accepts a request, changes status, and saves a comment. Each of those runs in one database transaction.
- RequestListService resolves the user's role to a FilterStrategy. That is the resolution step described in ADR-DESIGN-002.
- StaffFilterStrategy, RequesterFilterStrategy and ManagementFilterStrategy stay as separate classes. Staff rules were not copied into the requester class.
- StatusTransitionGuard remains the single table of legal steps and of who may perform them. The service asks the guard, then writes the row.
- New response shapes are records in api/dto, the same style as RequestResponse.

The paths added on the existing /api/v1/requests controller are: GET /requests for the scoped list, GET /requests/{id} for the detail, POST /requests/{id}/accept to take an Open request, POST /requests/{id}/status for the next step, and POST /requests/{id}/comments for a note.

6. How the staff view uses the same look

The signed-in shell is unchanged: navy header, CivicConnect title, name and role, Sign out, and the white navigation bar. Staff do not get a different header.

Both staff screens are a white card with the same radius, shadow and page title colour as Submit a request and the login card. Filters and the comment box use the same labels, inputs, selects and 44px-tall controls. Errors use alert-error. Success uses alert-success. The secondary actions (Back to requests, Save comment) use btn-outline.

New pieces that the submit form did not need, such as the request rows and the status pills, use the same tokens from app.css: navy text, the #000080 button colour, the grey border, and the page

background. Resolved uses the same green as the success alert. Nothing introduces a second colour system.

The layout stays inside the existing 960px column. On a narrow phone the filters wrap, the header stacks, and the action buttons become full width, using the same 480px breakpoint already in app.css. Request rows are stacked cards rather than a wide table, so the list does not force sideways scrolling.

7. What a staff member actually does

Sign in as staff@civicconnect.local. The Requests screen opens immediately.

- The introduction under the title states the scope: open requests you can accept, and requests assigned to you.
- Each row shows the request number, status, title, category, date submitted, and the assignee or the word Unassigned.
- Search matches the number, title, category, status and assignee name. Status and category filters and the date sort can be combined.
- Choosing a row opens the detail. Back to requests, or the Requests nav item, returns to the list and reloads it.
- The detail shows every field the requester submitted, plus who submitted it, who it is assigned to, when it was submitted and when it was last updated.
- Status history lists each successful change: the old status, the new status, who changed it, and when. A failed or illegal attempt does not add a line.
- Comments are listed under the history. A new comment is checked in the browser and again on the server. The limit is 2000 characters, the same ceiling as a request description.

8. What happens when Accept or the next status is pressed

The browser sends the request number and the version number it last loaded.

The service loads the signed-in user, loads the request, and checks that this user is allowed to see it. If the version on screen is older than the row in the database, the service stops with HTTP 409 and the message "This request was updated by someone else. Reload and try again." The screen then reloads the request. That is the optimistic lock already on the service_requests table (@Version), which the risk register describes for two staff members trying to claim one request.

Accept is refused unless the user is Staff and the status is still Open. It sets the assignee to that staff member, sets the status to Assigned, updates the timestamp, and inserts one status_history row in the same transaction. If any part fails, none of it is kept.

A later status change is refused with HTTP 400 when the step is not the next legal one, and with HTTP 403 when the step is legal but this person is not the assignee. A requester cannot call these actions. The history row is written only after those checks pass.

The status history row is written by ServiceRequestService in that same transaction. AuditLogObserver is still a placeholder class, and StatusTransitionGuard's own comment says the caller is responsible for the write. The detail screen reads those history rows, so the action history in FR007 is the status_history table plus the comments table.

9. What this does not change

- Requester sign-in and Submit a request are unchanged. A requester still does not see the staff list.
- Management still has the temporary home screen. Management filtering and the summary counts (FR012 and FR013) are not this view.
- Staff cannot assign a request to a different staff member. Management reassignment in D6 is a management action and is not on this screen.
- The requester home, My Requests and the shared RequestDetail placeholder files were left as they were.

10. How to try it

- Start the backend from src/backend and the frontend from src/frontend (npm run dev).
- Sign in as requester@civicconnect.local with password Password123! and submit a request.
- Sign out. Sign in as staff@civicconnect.local with the same password.
- Find the new request, filter or sort the list, open it, accept it, add a comment, then use Start work.
- The status pill and the history should show each step. Resolve and Close appear only after the previous step.